R4DS 01 – Introduction & Workflow Basics

Introduction

What is Data Science?

- Transform **raw data** into understanding, insight, and knowledge
- This course follows *R for Data Science* (2e) by Wickham, Çetinkaya-Rundel & Grolemund
- Tools: R + tidyverse ecosystem
- **80/20 rule**: the tools here cover ~80% of every project

Import $\rightarrow$ **Tidy** $\rightarrow$ **Transform** $\leftrightarrow$ **Visualize** $\rightarrow$ **Communicate**

- **Import**: load data from files, databases, APIs into R
- **Tidy**: each column = variable, each row = observation
- **Transform**: filter, create new variables, summarize
- **Visualize**: discover patterns, raise new questions
- **Communicate**: share results with others
- **Program**: the cross-cutting tool used in every step

What This Course Covers (and Doesn't)

Covers:

- Data wrangling (import, tidy, transform)
- Visualization (ggplot2)
- Programming in R (functions, iteration)
- Communication (Quarto)

Does not cover:

- Modeling (see *Tidy Modeling with R*)
- Big data (see `data.table` for 10–100 GB)
- Other languages (Python, Julia)

Prerequisites

R and RStudio

R — the language

- Download from CRAN: `https://cloud.r-project.org`
- Recommend R $\geq$ 4.2.0

RStudio — the IDE

- Download from: `https://posit.co/download/rstudio-desktop/`
- Console pane (left): type and run R code
- Output pane (right): plots, help, files

The Tidyverse

An R **package** = collection of functions + data + documentation
Install the tidyverse (run once):

```
install.packages("tidyverse")
```

Load it (every session):

```
library(tidyverse)
```

Core packages: `ggplot2`, `dplyr`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, `forcats`, `lubridate`

Other Useful Packages

```
install.packages(
  c("arrow", "babynames", "curl", "duckdb",
    "gapminder", "ggrepel", "ggridges",
    "ggthemes", "hexbin", "janitor", "Lahman",
    "leaflet", "maps", "nycflights13",
    "openxlsx", "palmerpenguins", "repurrrsive",
    "tidymodels", "writexl")
)
```

If you see: `Error: there is no package called 'foo'` — just run `install.packages("foo")`

Coding Basics

Arithmetic

R works as a calculator:

```
1 / 200 * 30
```

```
#> [1] 0.15
```

```
(59 + 73 + 2) / 3
```

```
#> [1] 44.66667
```

```
sin(pi / 2)
```

```
#> [1] 1
```

Assignment with <-

Create objects with the **assignment operator** <-:

```
x <- 3 * 4
x
```

```
#> [1] 12
```

- Read as: "x **gets** 3 times 4"
- RStudio shortcut: **Alt** + - (inserts <- with spaces)
- The value is stored, not printed — type the name to see it

Vectors

Combine elements with `c()`:

```
primes <- c(2, 3, 5, 7, 11, 13)
primes
```

```
#> [1]  2  3  5  7 11 13
```

Arithmetic is **vectorized** (applied element-wise):

```
primes * 2
```

```
#> [1]  4  6 10 14 22 26
```

```
primes - 1
```

```
#> [1]  1  2  4  6 10 12
```

Comments and Naming

Comments

Everything after # is ignored by R:

```
# create vector of primes
primes <- c(2, 3, 5, 7, 11, 13)

# multiply primes by 2
primes * 2
```

```
#> [1]  4  6 10 14 22 26
```

Best practice: comment on *why*, not *what* or *how*

Object Names

Rules:

- Must start with a letter
- Can contain: letters, numbers, _, .

Convention — use **snake_case**:

```
i_use_snake_case       # recommended
otherPeopleUseCamelCase
some.people.use.periods
```

Case Sensitivity and Typos

R is **case-sensitive** and **exact**:

```
r_rocks <- 2^3
```

```
r_rock
#> Error: object 'r_rock' not found
R_rocks
#> Error: object 'R_rocks' not found
```

The contract: R does the tedious computation; you must be precise in your instructions.

Calling Functions

Function Syntax

General form:

```
function_name(argument1 = value1, argument2 = value2)
```

Example with `seq()`:

```
seq(from = 1, to = 10)
```

```
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

Named arguments can be shortened by position:

```
seq(1, 10)
```

```
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

Action	Shortcut
Assignment <-	Alt + -
Run current line	Ctrl + Enter
Tab completion	TAB
Command history	↑
Filter history	Ctrl + ↑
Keyboard shortcut list	Alt + Shift + K

Tip: RStudio auto-completes function names, arguments, and file paths. Type a few characters and press TAB.

Strings and Matching Pairs

```
x <- "hello world"
x
```

```
#> [1] "hello world"
```

Quotation marks and parentheses must come in **matched pairs**.
If you see + in the console, R is waiting for more input:

```
> x <- "hello
+
```

Press **Escape** to abort, then fix the missing " or).

Exercises

Exercise 1

Why does this code not work? Look carefully!

```
my_variable <- 10
my_varıable
```

```
#> Error: object 'my_varıable' not found
```

Exercise 1 — Solution

The second line uses a **Turkish dotless i** (`ı`) instead of a regular `i`.

Correct code:

```
my_variable <- 10
my_variable
```

```
#> [1] 10
```

Lesson: tiny typos matter. Train your eyes to spot differences.

Exercise 2

Fix each line so it runs correctly:

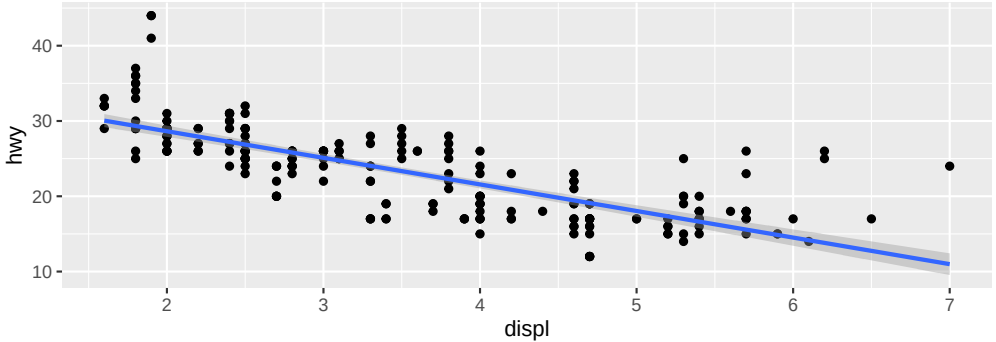
```
libary(todyverse)

ggplot(dTA = mpg) +
  geom_point(maping = aes(x = displ y = hwy)) +
  geom_smooth(method = "lm)
```

Exercise 2 — Solution

```
library(tidyverse)
```

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(method = "lm")
```



Fixes: `libary`→`library`, `todyverse`→`tidyverse`, `dTA`→`data`, `maping`→`mapping`, missing comma after `displ`, unclosed quote in `"lm"`, `aes()` moved to `ggplot()` so all layers inherit it.

Exercise 3

Press **Alt + Shift + K** (Windows/Linux) or **Option + Shift + K** (Mac).

What happens?

Answer: Opens the **Keyboard Shortcut Quick Reference** in RStudio. Same as: Help → Keyboard Shortcuts Help.

Exercise 4

Which plot is saved as `mpg-plot.png`? Why?

```
my_bar_plot <- ggplot(mpg, aes(x = class)) +
  geom_bar()
my_scatter_plot <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
ggsave(filename = "mpg-plot.png", plot = my_bar_plot)
```

Answer: The **bar plot** is saved, because `ggsave()` uses the `plot` argument explicitly. Without `plot =`, it defaults to the last displayed plot.